Universidad de Caldas
Facultad de Inteligencia Artificial e Ingenierías

Análisis y Diseño de Algoritmos — 2026-B

Proyecto semestral

Subproyecto 2 — Reparto de presupuesto y selección de participantes

Manual técnico de la solución base

Integrante: Juan Esteban Guzmán Henao

Profesora: Luz Enith Guerrero

Grupo: 2

Repositorio: `https://github.com/GuzGuz0208/ADA`

Versión: etiqueta `v0.1-primera-entrega`

Manizales, 6 de noviembre de 2026

Índice

Presentación **2**

1. Entorno **2**

2. Arquitectura **2**
2.1. Estructura de carpetas 2
2.2. Módulos de `src/ada_g2/` 2

3. Funcionamiento **3**
3.1. Flujo del programa por ronda 3
3.2. Algoritmo de la Parte 2 (`select.py`) 4
3.3. Algoritmo de la Parte 1 (`allocate.py`) 5
3.4. Certificados (`certificates.py`) 6

4. Uso paso a paso **6**
4.1. Formato del JSON de entrada 6
4.2. Pasos 7
4.3. Ejemplo de ejecución: caso de la profesora 8
4.4. Ejemplo de ejecución: instancia generada 8
4.5. Ejemplo de ejecución: subcomando `ajustada` 8
4.6. Significado de la salida 9
4.7. Códigos de salida 10

5. Validación y límites **10**
5.1. Pruebas disponibles 10
5.2. Certificados por ejecución 11
5.3. Razón frente al exacto 11
5.4. Errores conocidos 11
5.5. Supuestos y límites 11
5.6. Alcance: solución base frente a variantes 12

6. Identificación y enlace **12**

Presentación

Este manual describe el código de `src/ada_g2/` tal como está en la versión `v0.1-primera-entrega` (paquete 0.1.0). Su propósito es que otra persona entienda cómo está construida la implementación y pueda reproducir su ejecución sin explicaciones adicionales de los autores. Sigue el orden de la guía de la primera entrega (§6): Entorno, Arquitectura, Funcionamiento, Uso, Validación y límites, e Identificación y enlace.

Esta versión es la **solución base** de la primera entrega y **no es la solución final**: la Sección 5.6 separa lo implementado de lo parcial y de lo no desarrollado. Las demostraciones (NP-dureza, optimalidad del reparto, cota $1 - 1/e$) están en el informe técnico (`informe/main.tex`), no aquí. Los supuestos de modelado se identifican como D-xx (registro de decisiones del proyecto). Toda la salida de consola mostrada en este manual es salida real de ejecuciones del código.

1. Entorno

Elemento	Detalle
Lenguaje	Python $\geq$ 3.10 (probado con 3.12.10)
Dependencias de ejecución	`numpy>=1.26`, `scipy>=1.11` (`pyproject.toml`)
Desarrollo y pruebas	`matplotlib>=3.8`, `pytest>=8.0`, `hypothesis>=6.100` (`requirements.txt`)
Sistema compatible	Probado en Windows 11 (Git Bash y PowerShell). No usa rutas absolutas ni nada específico del sistema, por lo que debería funcionar en Linux y macOS, pero allí **no se ha probado**.
Instalación	`python -m venv .venv`, activar el entorno, `pip install -r requirements.txt`, `pip install -e .`
Punto de entrada	`python -m ada_g2 run|generar|ajustada` (`src/ada_g2/__main__.py` llama a `cli.main`)
Semilla por defecto	20261105 (`ada_g2.SEMILLA_POR_DEFECTO`), supuesto D-11

Activación del entorno: en Windows, `.venv\Scripts \activate` ; en Linux/macOS, `source .venv/bin/activate`.

2. Arquitectura

2.1. Estructura de carpetas

Carpeta	Responsabilidad
`src/ada_g2/`	el código (paquete instalable)
`tests/`	pruebas (7 archivos, 240 pruebas con parametrización)
`results/`	salidas por caso: `caso_<nombre>/`
`informe/`	informe técnico en LaTeX, con espejo en Markdown
`manual/`	este manual (fuente `manual_tecnico.tex`; versión PDF `manual_tecnico.pdf`; espejo `MANUAL_TECNICO.md`)
`project-docs/`	enunciado y JSON de la profesora (solo lectura)
`agent/`	memoria de trabajo de los agentes (decisiones D-xx, bitácora); no es parte de la solución

2.2. Módulos de `src/ada_g2/`

Módulo	Responsabilidad	Funciones y clases principales
`io.py`	Cargar y validar el JSON. Reúne *todos* los errores con su ruta (p. ej. `nodos[3].costo_participacion`) y lanza `EntradaInvalida`. Construye la matriz simétrica W con $W_{ii} = 1$.	`cargar`, `desde_dict`, `Instancia`, `EntradaInvalida`
`model.py`	Modelo matemático: utilidad $u(x) = a\ln(1+x)$, ganancia marginal, función de cobertura f y vector residual R_j.	`utilidad`, `ganancia_marginal`, `residual_inicial`, `ganancias_todas`, `ganancia_uno`, `actualizar_residual`, `f_directa`
`select.py`	**Parte 2.** Voraz por ganancia marginal (criterios `ganancia` y `razon`), mejor nodo individual y escogencia según el modo.	`seleccionar`, `voraz`, `mejor_singleton`, `limites`, `MODOS`
`allocate.py`	**Parte 1.** Reparto voraz con montículo (`heapq`) y topes por nodo.	`asignar`, `ResultadoReparto`, `PasoReparto`
`rounds.py`	Cadena de T rondas (Parte 2 y luego Parte 1), topes restantes, política de gasto, fallas con probabilidad q, verificación explícita de restricciones y comparación con el exacto.	`ejecutar`, `verificar_ronda`, `comparar_exacto`, `Corrida`, `Ronda`
`certificates.py`	Certificados por ejecución (D-14): dualidad de Lagrange para el reparto; cota por programa lineal (HiGHS) para la selección.	`certificar_reparto`, `certificar_seleccion`
`exact.py`	Referencias exactas solo para validar: fuerza bruta de la selección (DFS) y programación dinámica del reparto.	`fuerza_bruta`, `dp_reparto`, `num_subconjuntos`
`generator.py`	Instancias sintéticas con el mismo esquema JSON, deterministas por semilla; instancia ajustada D-15 (familia general con $N \neq M$).	`generar`, `ajustada`, `instancia_ajustada`
`trace.py`	Escribe `resultado.json`, `tiempo.csv`, `traza.md` y `traza.csv`.	`escribir_json`, `escribir_tiempos`, `escribir_traza`, `a_dict`
`cli.py`	Línea de comandos `run`, `generar` y `ajustada`; códigos de salida.	`main`, `cmd_run`, `cmd_generar`, `cmd_ajustada`

Dependencias entre módulos: `cli` → `io`, `rounds`, `trace`, `generator`; `rounds` → `select`, `allocate`, `certificates`, `exact`, `model`, `io`; `select` → `model`; `allocate` → `model`; `certificates` → `model`, `allocate`.

3. Funcionamiento

3.1. Flujo del programa por ronda

Para cada ronda $t = 1, \ldots, T$ el programa ejecuta el flujo de la Figura 1. Los números coinciden con la lista de pasos que sigue.

1. **Disponibles.** A_t = nodos con presupuesto total restante > 0 (D-21) que no fallan en la ronda; cada nodo falla con probabilidad q (D-07: $q = 0$ por defecto).

2. **Parte 2, selección.** `seleccionar` devuelve $S_t \subseteq A_t$ según el modo (`k`, `costo`, `ambos`).

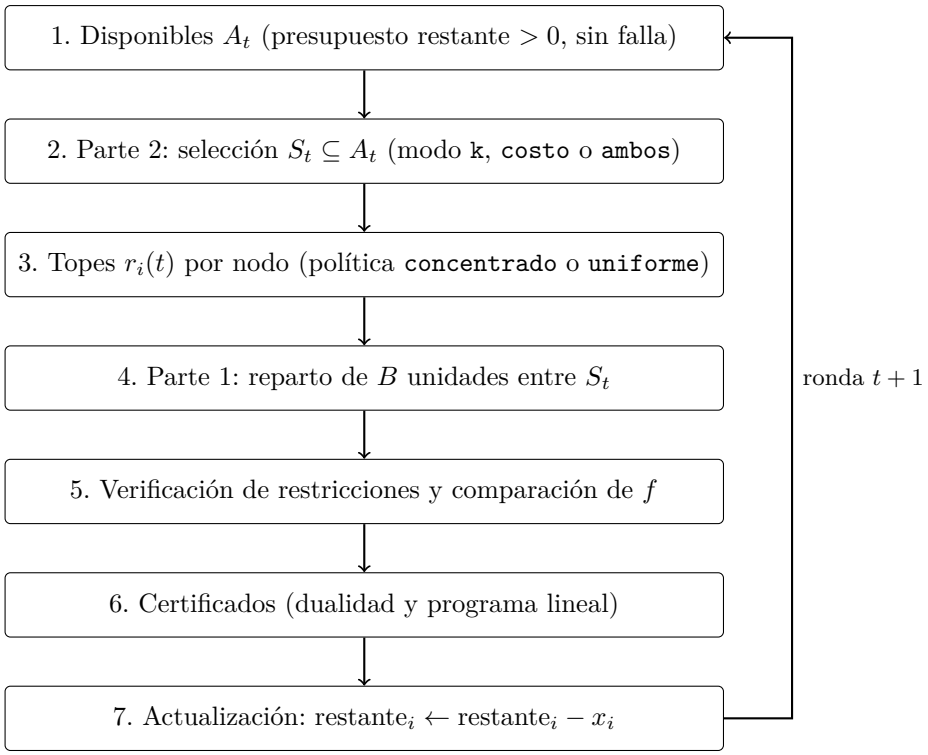


Figura 1: Pipeline por ronda. Al terminar las T rondas se verifica la restricción global.

3. **Topes.** $r_i(t)$ es el restante del nodo i (política `concentrado`) o $\lceil \text{restante}_i/(T-t+1) \rceil$ (política `uniforme`, D-22).

4. **Parte 1, reparto.** `asignar(a_S, topes_S, B)` reparte B unidades entre S_t.

5. **Verificación.** `verificar_ronda` comprueba las restricciones y se compara f incremental con f directa.

6. **Certificados.** Dualidad (reparto) y programa lineal (selección).

7. **Actualización.** restante$_i \leftarrow$ restante$_i - x_i$.

Al final se verifica la restricción global $\sum_t x_{i,t} \leq R_i$. La corrida es correcta (`ok`) si todo cumple.

3.2. Algoritmo de la Parte 2 (`select.py`)

Modelo. $f(S) = \sum_j a_j[1 - \prod_{i \in S}(1 - w_{ij})]$. Con el vector residual $R_j = \prod_{l \in S}(1 - w_{lj})$ (inicialmente $R_j = 1$), la ganancia de agregar $i \notin S$ es $\Delta(i \mid S) = \sum_j a_j \, w_{ij} \, R_j$. Al elegir i se actualiza $R_j \leftarrow R_j(1 - w_{ij})$ en $O(M)$.

Implementación. En cada paso `ganancias_todas` calcula el producto matriz-vector $W(a \odot R)$, es decir, las ganancias de *todos* los candidatos en $O(NM)$. Se enmascaran los ya elegidos, los que no son candidatos y los que no caben en el costo, y se escoge con `np.argmax` (primer máximo, es decir, menor índice, D-12).

Condiciones de parada. $|S| = k_{\text{máx}}$, o no hay candidato factible, o la mejor ganancia es ≤ 0.

Modos (`limites`): `k` impone $|S| \leq k$; `costo` impone $\sum c \leq L$; `ambos` impone las dos. En todos, $|S| \leq k$ significa "a lo sumo" (D-27).

Heurísticas. En modo `k` solo se usa `voraz_ganancia`, con garantía $1-(1-1/k)^k \geq 1-1/e$ (Nemhauser, Wolsey y Fisher, 1978; la prueba está en el informe, Sección 3.3). En los modos `costo` y `ambos` se toma el mejor entre `voraz_ganancia`, `voraz_razon` (ganancia/costo) y `singleton` (mejor nodo individual factible); en empate gana la primera en ese orden (D-23). El voraz factible salta a los candidatos que no caben en lugar de detenerse (D-29). **No se afirma garantía en `costo` ni en `ambos`** (R-01).

Costo. $\Theta(k\,N\,M)$, que es $\Theta(kN^2)$ con $M = N$ en el JSON de la profesora. Las evaluaciones útiles son $P = kN - k(k-1)/2$, pero el código calcula las N ganancias en cada paso y hace literalmente kNM multiplicaciones; la clase de complejidad no cambia (informe, Sección 3.2, nota de fidelidad).

Algoritmo 1 Voraz por ganancia marginal (Parte 2, modo `k`)

Precondición: matriz $W \in [0,1]^{N\times M}$, pesos $a \in \mathbb{R}^M_{>0}$, cupo k
Poscondición: S con $|S| \leq k$

$S \leftarrow \emptyset$; $R_j \leftarrow 1$ para todo j
mientras $|S| < k$ **hacer**
  $g \leftarrow W\,(a \odot R)$ ▷ ganancias de todos los candidatos, $O(NM)$
  $i^* \leftarrow \arg\max_{i\notin S} g_i$ ▷ empate: menor índice
  si $g_{i^*} \leq 0$ **entonces**
    salir
  $S \leftarrow S \cup \{i^*\}$; $R_j \leftarrow R_j\,(1 - w_{i^*j})$ para todo j
devolver S

3.3. Algoritmo de la Parte 1 (`allocate.py`)

La ganancia marginal de dar la unidad $(x+1)$ al nodo i es $g_i(x) = a_i \ln \frac{x+2}{x+1}$, estrictamente decreciente en x.

Estructura de datos. Montículo de máximos con `heapq`; como `heapq` es de mínimos, se guarda `(-g, i)`. Contiene, para cada nodo con tope > 0, su próxima ganancia. Se construye con `heapify` en $O(N)$.

Ciclo. Mientras se hayan asignado menos de B unidades y el montículo no esté vacío: se toma la mayor ganancia (la raíz), se incrementa `x[i]`; si `x[i] < tope[i]` se reemplaza la raíz por la nueva ganancia (`heapreplace`, $O(\log N)$) y si no se saca (`heappop`, $O(\log N)$). Desempate por menor índice (D-12).

Condición de parada. Se asignaron B unidades **o** el montículo quedó vacío (D-28: si $\sum$ topes $< B$, sobra presupuesto y no es un error).

Invariante. Tras j pasos, las j unidades tomadas son las j mayores ganancias marginales del multiconjunto; por eso el resultado es óptimo (Teorema 3.1 del informe, por intercambio y concavidad).

Costo. $T(N,B) = O(N + B\log N)$. La utilidad final se recalcula directamente, $\sum a_i \ln(1+x_i)$, y no se acumula.

Algoritmo 2 Reparto voraz con montículo (Parte 1)

Precondición: nodos de S con coeficientes $a_i > 0$, topes $r_i \geq 0$, presupuesto $B \in \mathbb{N}$
Poscondición: $x_i \in \mathbb{N}$, $0 \leq x_i \leq r_i$, $\sum_i x_i \leq B$, utilidad máxima

$x_i \leftarrow 0$ para todo i; $H \leftarrow$ montículo con $(a_i \ln 2,\ i)$ para cada i con $r_i > 0$
$b \leftarrow 0$
mientras $b < B$ **y** $H \neq \emptyset$ **hacer**
    $(g, i) \leftarrow$ raíz de H ▷ mayor ganancia; empate: menor índice
    $x_i \leftarrow x_i + 1$; $b \leftarrow b + 1$
    si $x_i < r_i$ **entonces**
        reemplazar la raíz por $(a_i \ln \frac{x_i+2}{x_i+1},\ i)$
    si no
        sacar la raíz
devolver x

3.4. Certificados (`certificates.py`)

Reparto (dualidad). Con λ igual a la ganancia de la última unidad asignada (0 si los topes impiden gastar B), la cota $UB(\lambda) = \lambda B + \sum_i \max_{0 \leq x \leq r_i}[a_i \ln(1+x) - \lambda x]$ es mayor o igual que el óptimo. El programa exige que la brecha UB − utilidad sea $\leq 10^{-9}$. Si se cumple, el reparto es óptimo sin conocer el óptimo por otro método.

Selección (programa lineal). Para cualquier S factible y f monótona y submodular, $f(S^*) \leq f(S) + \max\{\sum_e \Delta(e \mid S) z_e : \sum z_e \leq k,\ \sum c_e z_e \leq L,\ 0 \leq z \leq 1\}$. Se resuelve con `scipy.optimize.linprog` (HiGHS). La razón certificada es $f(S)$/cota. Vale con cupo y costo a la vez.

Exactos de referencia (`exact.py`, solo validación). `fuerza_bruta`: DFS sobre subconjuntos factibles (hasta 3 000 000 de visitas). `dp_reparto`: programación dinámica sobre nodos y presupuesto.

4. Uso paso a paso

4.1. Formato del JSON de entrada

Esquema de la profesora (`project-docs/entrada_G2_red_15_nodos.json`). Todos los campos son obligatorios.

```
{
  "parametros": {
    "numero_nodos": 15,
    "numero_rondas": 3,
    "cupo_participantes_por_ronda_k": 5,
    "presupuesto_ruido_por_ronda_B": 30,
    "presupuesto_costos_participacion_por_ronda": 24
  },
  "grafo": [
    { "u": "N01", "v": "N02", "peso_influencia": 0.25 }
  ],
  "nodos": [
    { "id": "N01", "coeficiente_utilidad": 8.5,
      "presupuesto_total_nodo": 18, "costo_participacion": 3 }
  ]
}
```

Campo	Tipo y regla	Significado en el código
`parametros.numero_nodos`	entero ≥ 1; debe coincidir con `len(nodos)`	N
`parametros.numero_rondas`	entero ≥ 1	T
`parametros.cupo_participantes_por_ronda_k`	entero ≥ 1	k. Si $k \geq N$ se emite un aviso (el enunciado supone $k < N$); no es error
`parametros.presupuesto_ruido_por_ronda_B`	entero ≥ 0	B: unidades a repartir por ronda
`parametros.presupuesto_costos_participacion_por_ronda_L`	entero ≥ 0	L: costo máximo por ronda
`grafo[]`	lista de aristas	cada par se lista una vez; se trata como no dirigido (D-10): $w_{uv} = w_{vu}$
`grafo[].u`, `grafo[].v`	ids de `nodos`; sin autolazos	extremos de la arista
`grafo[].peso_influencia`	real en $[0, 1]$; una arista repetida debe tener el mismo peso	w_{uv}. Pares ausentes: $w = 0$. La diagonal $w_{ii} = 1$ la fija el modelo (D-02)
`nodos[].id`	texto no vacío, único	nombre; el orden de la lista define el índice y el desempate (D-12)
`nodos[].coeficiente_utilidad`	real > 0	a_i (en f, a_j; en u, a_i)
`nodos[].presupuesto_total_nodo`	entero ≥ 0	R_i: tope de ruido en las T rondas (D-06)
`nodos[].costo_participacion`	entero ≥ 0	c_i

Cualquier incumplimiento produce el código de salida 2 y un mensaje que lista todos los problemas, por ejemplo: `grafo[0].peso_influencia: debe estar en [0,1], se recibio 1.5`. La lógica no contiene constantes del caso: otro JSON con este esquema se ejecuta sin cambiar código.

4.2. Pasos

1. **Instalar el entorno** (Sección 1):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
pip install -e .
```

2. **(Opcional) generar una instancia** sintética con el mismo esquema:

```
python -m ada_g2 generar --n 6 --rondas 2 --k 3 --B 10 --L 12 --seed 7 --out ejemplo_n6.
    json
```

   Parámetros de `generar`: `-n` (obligatorio), `-rondas` (3), `-k` (5), `-B` (30), `-L` (por defecto `round(0.8*k*costo medio)`), `-seed` (20261105), `-densidad` (1.0 = grafo completo; con valor menor, cada par aparece con esa probabilidad), `-out` (obligatorio).

3. **Ejecutar** un caso:

```
python -m ada_g2 run --input <json> --out <carpeta> [--modo ambos|k|costo]
    [--seed S] [--repeticiones R] [--q Q] [--politica concentrado|uniforme]
```

4. **Leer la salida** en consola (resumen por ronda y veredicto) y los archivos en `<carpeta>/caso_<nombre del json>/`.

5. **Revisar el código de salida** (`echo $?` en Bash o `echo %ERRORLEVEL%` en cmd): 0, 2 o 3 (Sección 4.7).

6. **(Opcional) verificar la cota** con el subcomando `ajustada`:

```
python -m ada_g2 ajustada [--k 2 3 5]
```

4.3. Ejemplo de ejecución: caso de la profesora

Comando (salida real de consola; la última ruta se abrevió a `out_manual`):

```
python -m ada_g2 run --input project-docs/entrada_G2_red_15_nodos.json --out out_manual

Caso entrada_G2_red_15_nodos: N=15 T=3 k=5 B=30 L=24 modo=ambos
  ronda 1: S=['N09', 'N11', 'N13', 'N14', 'N15'] f=145.5795 costo=23 reparto={'N09': 7, 'N11':
     5, 'N13': 5, 'N14': 7, 'N15': 6} u=88.5640
  ronda 2: S=['N09', 'N11', 'N13', 'N14', 'N15'] f=145.5795 costo=23 reparto={'N09': 7, 'N11':
     5, 'N13': 5, 'N14': 7, 'N15': 6} u=88.5640
  ronda 3: S=['N09', 'N11', 'N13', 'N14', 'N15'] f=145.5795 costo=23 reparto={'N09': 4, 'N11':
     8, 'N13': 8, 'N14': 4, 'N15': 6} u=85.6108
  suma f=436.7386  suma u=262.7387  pipeline (mediana)=5.618 ms
  restricciones y certificados: TODO CUMPLE
  salida: out_manual\caso_entrada_G2_red_15_nodos
```

El tiempo (`pipeline (mediana)`) varía de una máquina a otra; los demás valores son deterministas. Las cifras de referencia del caso están en `results/caso_entrada_G2_red_15_nodos/` y se discuten en la Sección 4 del informe.

4.4. Ejemplo de ejecución: instancia generada

Con los pasos 2 y 3 (con `-seed 7`) se obtuvo, en consola:

```
escrito ejemplo_n6.json (N=6, aristas=15, semilla=7)
Caso ejemplo_n6: N=6 T=2 k=3 B=10 L=12 modo=ambos
  ronda 1: S=['N01', 'N04', 'N06'] f=48.8055 costo=12 reparto={'N01': 3, 'N04': 5, 'N06': 2} u
    =44.7612
  ronda 2: S=['N01', 'N04', 'N06'] f=48.8055 costo=12 reparto={'N01': 3, 'N04': 5, 'N06': 2} u
    =44.7612
  suma f=97.6109  suma u=89.5223  pipeline (mediana)=3.366 ms
  restricciones y certificados: TODO CUMPLE
  salida: salida\caso_ejemplo_n6
```

En la ronda 1, `traza.md` muestra que el voraz por ganancia elige N04 (ganancia 38.223000), luego N06 (7.988310) y N01 (2.594150), con costo acumulado 2, 7 y 12 (límite $L = 12$).

4.5. Ejemplo de ejecución: subcomando `ajustada`

Salida real de consola; sale con código 0 porque la razón observada coincide con $1-(1-1/k)^k$ para todos los k:

```
python -m ada_g2 ajustada

k | N x M | voraz S | f(voraz) | f(optimo) | razon | 1-(1-1/k)^k
2 | 4 x 8 | [0, 1] | 6 | 8 | 3/4 = 0.75000 | 3/4
3 | 6 x 81 | [0, 1, 2] | 57 | 81 | 19/27 = 0.70370 | 19/27
5 | 10 x 15625 | [0, 1, 2, 3, 4] | 10505 | 15625 | 2101/3125 = 0.67232 | 2101/3125
```

```
cota alcanzada exactamente
```

4.6. Significado de la salida

Cada `run` escribe cuatro archivos en `<carpeta>/caso_<nombre>/`.

`resultado.json`. Campos principales:

Campo	Contenido
`version`, `caso`, `entorno`	versión del paquete, nombre del caso, versión de Python y plataforma
`parametros`	N, T, k, B, L, aristas, modo, q, política, semilla
`avisos`	lista de avisos de la carga (p. ej. $k \geq N$)
`rondas[]`	una entrada por ronda (ver abajo)
`totales`	`suma_f`, `suma_utilidad`, `gasto_por_nodo`, `presupuesto_total_nodo`
`verificacion_global`	$\sum_t x_{i,t} \leq R_i$
`todo_cumple`	`true` si todo cumple (código de salida 0)

Cada elemento de `rondas[]` contiene:

Campo	Contenido
`ronda`, `disponibles`	número de ronda; nodos disponibles A_t
`seleccion`	ids de S_t
`detalle_seleccion`	`elegida` y `alternativas` (heurística, S en orden de elección, f y costo)
`f`, `costo_participacion`	valor de $f(S_t)$ y costo $\sum c$ de S_t
`topes_antes`, `topes_efectivos`	restante por participante antes de la ronda; tope que aplica la política
`reparto`, `unidades_asignadas`, `gasto_ruido`	unidades por nodo; total asignado; ruido gastado
`utilidad`	$\sum a_i \ln(1 + x_i)$ de la ronda
`verificacion_restricciones`	lista de objetos con `restriccion, descripcion, valor, limite, aplica, cumple`
`certificado_reparto`	`lambda`, `cota_superior`, `utilidad`, `brecha`, `tolerancia`, `ok`
`certificado_seleccion`	`f_S`, `extra`, `cota_superior`, `razon_certificada`, `ok`
`comparacion_exacto`	`seleccion_optima`, `f_optima`, `razon_voraz_sobre_optimo`, `subconjuntos_evaluados`, `subconjuntos_teoricos`, `reparto_dp_utilidad`, `reparto_voraz_utilidad`, `reparto_dif_abs`, `reparto_optimo`. Es `null` si $N > 20$; `reparto_dp_utilidad` es `null` si $\lvert S\rvert B^2 > 5 \cdot 10^7$

`tiempo.csv`. Columnas `fase, repeticiones, mediana_s, minimo_s, maximo_s` (segundos, 9 decimales). Fases: `carga_json`; `seleccion` y `reparto` (suma de las T rondas); `verificacion`; `certificados`; `seleccion_r<t>` y `reparto_r<t>` por ronda; `pipeline_total`; `referencia_exacta (1 corrida)` (fuerza bruta y DP, una sola vez; no entra en el pipeline). Se ejecuta una corrida de calentamiento sin medir y luego `-repeticiones` corridas; se reporta la mediana (D-11). Los certificados cuestan más que el algoritmo mismo (programa lineal): en el caso de la profesora, 5.2023 ms de 6.6853 ms.

`traza.md`. Por ronda: disponibles; para cada heurística, una tabla por paso (`paso, elegido, ganancia marginal, puntaje, costo acum., f acum.`, más la lista de ganancias de los candidatos factibles, truncada a 10); el reparto unidad a unidad (`unidad, nodo, x antes, ganancia marginal, utilidad acum.`); la tabla de verificación de restricciones; y las líneas de los certificados y del exacto. Cierra con la verificación global.

`traza.csv`. Columnas `ronda, fase, heuristica_o_restriccion, paso, nodo, valor, detalle`. Valores de `fase`: `seleccion` (nodo elegido; `valor` es la ganancia y `detalle` trae costo y f acumulados), `seleccion_candidato` (ganancia de cada candidato evaluado en ese paso), `reparto` (`valor` es la ganancia; `detalle` trae x antes y la utilidad acumulada), `verificacion` (una fila por restricción; ronda 0 es la global) y `certificado`.

4.7. Códigos de salida

Código	Significado
0	todo cumple
2	entrada inválida: archivo inexistente, JSON mal formado o esquema incumplido
3	falla una restricción o un certificado (en `ajustada`: la razón no coincide con la cota)

El código 3 en `run` no está cubierto por una prueba automática aún (no probado).

5. Validación y límites

5.1. Pruebas disponibles

Se ejecutan con `pytest -q`. Resultado verificado al escribir este manual: **240 passed** en unos 18 s (Python 3.12.10).

Archivo	Qué prueba
`test_reparto.py`	el voraz coincide con la DP exacta en ≥ 500 instancias (700 en total); restricciones ($\sum x \leq B$, $0 \leq x \leq$ tope, entero); certificado de dualidad (brecha $\leq 10^{-9}$)
`test_funcion_f.py`	$f(\emptyset) = 0$, monotonía, submodularidad (600 instancias), ganancia incremental igual a la diferencia directa, caso Cobertura Máxima
`test_seleccion.py`	restricciones en los tres modos; sin nodos repetidos; razón voraz/exacto $\geq 1 - (1 - 1/k)^k$ en modo `k` (con `hypothesis`); certificado LP mayor o igual que el óptimo; instancia ajustada D-15 (razones exactas 3/4, 19/27, 2101/3125 para $k = 2, 3, 5$); contraejemplos (voraz no óptimo, falla de submodularidad, falla con costos); conteo de evaluaciones P y kNM; regresión BUG-1; subcomando `ajustada`
`test_np_reduccion.py`	la reducción desde Cobertura Máxima: ejemplo 4×5 del informe, instancia negativa, equivalencia CobMax-Dec $\Leftrightarrow$ Sel-Dec en la subfamilia $M = N$, $w_{ii} = 1$, $a > 0$ (60 instancias)
`test_caso_profesora.py`	valores de referencia del caso (selección, f, exacto en modo `k`, reparto) re-derivados con el código
`test_cli_validacion.py`	13 JSON inválidos con salida 2 y mensaje; 5 válidos con salida 0; CLI sobre JSON sintéticos

Archivo	Qué prueba
`test_smoke.py`	humo: reparto igual a la DP, el caso de la profesora cumple las restricciones, voraz frente a fuerza bruta en una instancia generada, generador determinista, mensajes de validación

5.2. Certificados por ejecución

Cada corrida falla (código 3) si algún certificado no se cumple: dualidad del reparto (brecha $\leq 10^{-9}$) o cota LP de la selección (cota $\geq f(S)$). Para el caso de la profesora: brecha 0 en las tres rondas, cota 151.9871 y razón certificada 0.9578. El subcomando `ajustada` corre voraz y fuerza bruta sobre la instancia ajustada (D-15) y comprueba que la razón es exactamente $1-(1-1/k)^k$.

5.3. Razón frente al exacto

En el caso de la profesora (modo `ambos`) la razón voraz/óptimo es 1.0000 (`resultado.json`). En modo `k` es 0.99775, con óptimo 145.9072. Medido por el verificador (3 familias $\times$ 250 instancias, modo `k`): razón mínima 0.8885, siempre $\geq 1-(1-1/k)^k$; en `costo` 0.8864 y en `ambos` 0.8148, **sin garantía teórica** (R-01).

5.4. Errores conocidos

- **Ninguno abierto en la lógica.** BUG-1 (la selección usaba la longitud de a, es decir M, como número de candidatos N en `voraz` y `mejor_singleton`; fallaba con $N \neq M$ en la familia general) se corrigió usando `W.shape[0]`. No afectaba al caso de la profesora ($M = N$) y tiene prueba de regresión.
- **Ids repetidos.** Un id repetido en `nodos` genera además errores en cascada de "arista a nodo inexistente" que pueden ocultar otros mensajes (mejora pendiente en `io.py`).
- **Código de salida 3** en `run`: sin prueba automática.

5.5. Supuestos y límites

1. **Saturación del caso (R-02).** El grafo de la profesora es completo con pesos ≥ 0.2: $f(S)$ alcanza 145.5795 de un máximo $\sum a = 147.8$ (0.9850) y las heurísticas dan valores muy cercanos. El caso discrimina poco; la cota $1-1/e$ se verifica en familias sintéticas.
2. **Sin garantía con cupo y costo a la vez (R-01).** La garantía $1-(1-1/k)^k$ es solo para el modo `k`. En `ambos` se usa el mejor de tres heurísticas y se certifica a posteriori (razón certificada por LP), pero no hay cota a priori demostrada.
3. **Fuerza bruta solo para** $N \leq 20$ (`comparar_exacto`, `max_n = 20`; además se corta a los 3 000 000 de visitas). Con N mayor, `comparacion_exacto` es `null` y el tiempo de referencia exacta no se mide.
4. **DP del reparto omitida** si $|S|B^2 > 5 \cdot 10^7$ (D-24); entonces `reparto_dp_utilidad`, `reparto_dif_abs` y `reparto_optimo` son `null`. El certificado de dualidad sigue disponible.
5. **Reparto pseudo-polinomial en** B: $O(N + B \log N)$ es lineal en el valor de B, no en sus bits. Con B enorme y sin topes activos ni fallas es costoso (informe, Sección 1.3); el caso continuo con *water-filling* está esbozado, no implementado.
6. **El modelo es una decisión nuestra** (D-01, D-02): el enunciado no define u_i ni f. La profesora puede preferir otras formas.

7. **Nodos y topes**: un nodo con tope restante 0 deja de ser candidato (D-21); si $\sum$ topes $< B$ el ciclo termina sin gastar todo (D-28).

8. **Aristas no dirigidas** (D-10), semilla por defecto 20261105 (D-11), desempate por menor índice (D-12), $q = 0$ por defecto (D-07).

5.6. Alcance: solución base frente a variantes

Esta versión (`v0.1-primera-entrega`) es la **solución base** de la primera entrega y **no es la solución final**.

Componente	Estado en esta versión
Parte 1, reparto voraz exacto con montículo y topes por nodo	implementado, probado contra DP exacta y certificado por dualidad
Parte 2, selección voraz (modos `k`, `costo`, `ambos`), fuerza bruta de referencia y cota LP a posteriori	implementado y probado
T rondas en cadena (Parte 2 y luego Parte 1) con topes que bajan entre rondas	implementado ($q = 0$ en el caso de la profesora, D-07)
Variante (a), costos de participación con garantía demostrada (Khuller et al.)	**parcial**: hay mejor-de-3 (voraz, razón y mejor nodo único), pero sin garantía para cupo y costo (R-01) ni enumeración parcial
Variante (b), fallas con probabilidad q y comparación de políticas con ≥ 100 semillas	**parcial**: `-q` y `-politica` existen; falta el experimento comparativo
Variante (c), nodos relacionados (caso aditivo y caso competencia)	**no desarrollada**
Validación experimental (≥ 5 tamaños, gráfica tiempo frente a n contra $T(n)$)	**no desarrollada** (tarea T13)

Componentes aún no desarrollados. Barrido experimental por tamaños y estimación empírica de $T(n)$ (T13); deducción completa de $T(n)$ con constantes; variante (c); experimento comparativo de la variante (b). El informe y este manual se compilan en letX (D-20).

6. Identificación y enlace

- **Curso:** Análisis y Diseño de Algoritmos 2026-B. **Profesora:** Luz Enith Guerrero. **Universidad de Caldas.**
- **Subproyecto 2 (Grupo 2):** Reparto de presupuesto y selección de participantes.
- **Integrante:** Juan Esteban Guzmán Henao (trabajo individual).
- **Repositorio:** `https://github.com/GuzGuz0208/ADA`, rama `main`, etiqueta `v0.1-primera-entrega`.
- **Informe:** `informe/main.tex` (y `informe/INFORME.md`). **Resultados:** `results/caso_entrada_G2_red_15_r`